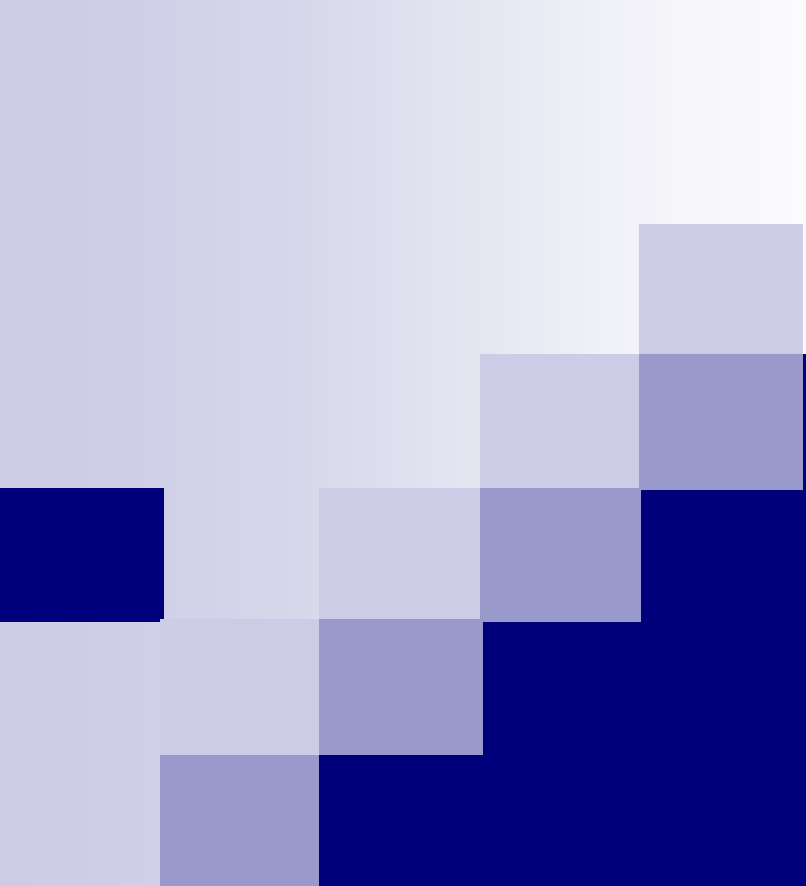




CH9. 빅데이터 플랫폼

전성환

72160261@dankook.ac.kr



■ 학습 내용

9.1 Helm이란?

9.2 Helm Chart 구조

9.3 나만의 Chart 만들기

9.4 values.yaml

9.5 Helm 라이프사이클

9.6 실습

9.1 Helm이란?

- Kubernetes 애플리케이션을 위한 패키지 매니저
 - yum, apt-get, npm처럼 K8s 리소스를 묶어서 관리
- Helm 구성 요소
 - Chart : 애플리케이션 배포에 필요한 K8s YAML 파일들의 묶음
 - Repository : Chart를 저장/공유하는 저장소
 - Release : 클러스터에 배포된 Chart의 인스턴스

9.1 Helm이란?

kubectl 만 쓰면?

- deployment.yaml 따로
- service.yaml 따로
- configmap.yaml 따로
- yaml 5~10개 일일이 apply
- 환경마다 값 직접 수정
- 버전 관리 / 롤백 어려움

Helm 을 쓰면?

- 한 묶음(Chart)으로 관리
- 한 줄로 설치/삭제
- values.yaml로 환경별 설정
- 버전(릴리즈) 관리 자동
- 롤백 한 줄이면 끝
- 팀/공개 저장소로 공유 쉬움

9.1 Helm이란?

- 쿠버네티스에 뭔가 설치할 때 **yaml**이 너무 많다
 - 간단한 웹 서비스 하나 → yaml 5~6개 (Deployment, Service, ConfigMap ...)
 - DB나 모니터링 시스템 → yaml 20~30개
 - 이걸 매번 직접 만들고 관리하는 건 비현실적
- **Helm**은 쿠버네티스의 "패키지 매니저"
 - Ubuntu의 apt-get install 처럼
 - macOS의 brew install 처럼
 - Python의 pip install 처럼
 - 필요한 것을 한 줄 명령어로 설치 / 삭제 / 업데이트
- **Helm**이 해주는 일
 - 여러 yaml을 하나의 패키지(**Chart**)로 묶어 관리
 - 환경마다 다른 값을 깔끔하게 분리
 - 설치한 것의 버전 / 이력 / 롤백을 자동 추적

9.1 Helm이란?

- Helm에서 자주 헛갈리는 용어
- Chart : 설치 패키지 (설계도)
 - 애플리케이션을 배포하기 위한 yaml 묶음
 - Ubuntu의 .deb 파일 같은 개념
 - 예) nginx-chart, prometheus-chart
- Repository : Chart들이 모여있는 저장소
 - Chart를 다운로드 받을 수 있는 곳 (인터넷 또는 사내)
 - Ubuntu의 apt 저장소 같은 개념
 - 예) bitnami, prometheus-community, grafana
- Release : 실제로 클러스터에 설치된 결과물
 - Chart를 설치하면 "Release"가 만들어짐
 - 같은 Chart도 여러 번 설치하면 Release가 여러 개 생김
 - 예) my-app, my-app-staging, my-app-prod (모두 같은 Chart)

9.1 Helm이란?

■ install vs upgrade

- install : 처음 설치할 때 (같은 이름이 있으면 에러)
- upgrade : 이미 설치된 것의 값 / Chart 변경

■ uninstall vs rollback

- uninstall : 릴리즈를 완전히 삭제 (모든 리소스 제거)
- rollback : 이전 버전으로 되돌림 (설치 상태는 유지)
- 실수로 잘못 업그레이드 했을 때는 **uninstall** 이 아니라 **rollback**

■ --set vs -f

- --set : 명령어에 직접 값 입력 (값이 적을 때)
- -f : 별도 파일로 값 관리 (값이 많거나 환경 분리할 때)
- 두 옵션을 동시에 쓸 수 있음 (--set 이 우선)

9.2 Helm Chart 구조

■ Chart는 디렉터리 하나로 이루어진 패키지

```
mychart/
├── Chart.yaml           # Chart 메타데이터 (이름, 버전, 설명)
├── values.yaml          # 사용자가 바꿀 수 있는 기본 설정값
├── charts/              # 의존하는 다른 Chart 저장 위치
├── templates/           # K8s 리소스 YAML 템플릿
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── _helpers.tpl     # 재사용 가능한 템플릿 조각
│   └── NOTES.txt        # 설치 후 안내 메시지
└── .helmignore          # 패키징 시 제외할 파일
```

● 핵심 3가지

Chart.yaml → 이 패키지가 무엇인지

values.yaml → 사용자가 어떻게 바꿀 수 있는지

templates/ → 실제로 무엇을 배포하는지

9.2 Helm Chart 구조

Chart.yaml

```
apiVersion: v2
name: mychart
description: My first Helm chart
type: application
version: 0.1.0      # Chart 버전
appVersion: "1.16.0" # 앱 버전
```

values.yaml

```
replicaCount: 1

image:
  repository: nginx
  tag: "1.16.0"

service:
  type: ClusterIP
  port: 80
```



Helm = 템플릿 + 값을 합쳐서 K8s에 배포해주는 도구

9.3 나만의 Chart 만들기

- helm create 명령어 한 줄로 기본 구조가 생성된다

```
helm create mychart
```

- 생성된 디렉터리 확인

```
tree mychart
mychart/
├── Chart.yaml
├── charts/
├── values.yaml
├── templates/
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── tests/
│       └── test-connection.yaml
```

- nginx를 배포하는 기본 Chart가 자동 생성됨 (바로 install 가능)

9.3 나만의 Chart 만들기

- 설치 전에 문법 체크

```
helm lint mychart
```

- 실제로 어떤 YAML이 만들어지는지 미리 보기

```
helm template mychart
```

- 설치 시 어떤 일이 일어날지 시뮬레이션 (--dry-run)

```
helm install my-release mychart --dry-run --debug
```

9.4 values.yaml

- 배포할 때 환경마다 값이 다를 때
 - 개발 환경 : 서버 1대, 적은 메모리
 - 운영 환경 : 서버 5대, 큰 메모리
 - 같은 앱인데 환경마다 yaml을 새로 짜야 할까?
- Helm의 해결책 : Chart는 그대로, values만 바꾼다
 - 템플릿(templates/) → 변하지 않는 구조
 - 값(values.yaml) → 환경마다 다른 부분만 모아둠
- 비유 : 이력서 양식과 작성 내용
 - 이력서 양식(templates)은 누구나 똑같음
 - 이름, 학력, 경력(values)은 사람마다 다름
 - 양식을 매번 새로 그릴 필요 없이 빈칸만 채우면 됨

9.4 values.yaml

- **--set** 옵션 : 명령어 한 줄로 값 변경
 - Chart는 그대로 두고, 일부 값만 다르게 설치할 때 사용

- 기본 사용법

```
# values.yaml 의 replicaCount 를 3으로 변경하여 설치
helm install my-app ./mychart --set replicaCount=3
```

- 여러 값을 동시에 변경

```
helm install my-app ./mychart \
  --set replicaCount=3 \
  --set image.tag=2.0.0 \
  --set service.type=NodePort
```

- 언제 쓰면 좋을까?

- 간단한 테스트, 임시 설정 변경
- 값이 1~3개로 적을 때
- 값이 많아지면 다음 슬라이드의 **-f** 옵션 사용 권장

9.4 values.yaml

- -f 옵션 : 별도 파일로 값 관리

- 환경마다 다른 값을 파일로 분리해서 관리
- 재사용, 버전 관리, 협업에 유리

- 환경별 values 파일 작성

values-dev.yaml (개발 환경)

replicaCount: 1

image:

tag: "latest"

resources:

limits:

memory: "256Mi"

values-prod.yaml (운영 환경)

replicaCount: 5

image:

tag: "1.30"

resources:

limits:

memory: " 300Mi "

- 설치 시 -f 로 값 파일 지정

```
helm install my-app ./mychart -f values-dev.yaml
```

```
helm install my-app ./mychart -f values-prod.yaml
```

9.5 Helm 라이프사이클 (1) - install

■ Chart 설치 및 상태 확인

로컬 Chart 디렉터리로 설치

```
helm install my-app ./mychart
```

values 일부만 덮어쓰기

```
helm install my-app ./mychart --set replicaCount=3
```

values 파일로 덮어쓰기

```
helm install my-app ./mychart -f custom-values.yaml
```

■ 설치된 릴리즈 확인

```
helm list
```

NAME	NAMESPACE	REVISION	STATUS	CHART	APP VERSION
my-app	default	1	deployed	mychart-0.1.0	1.16.0

```
helm status my-app
```

9.5 Helm 라이프사이클 (2) - upgrade & rollback

■ 릴리즈 업그레이드 (값 또는 Chart 변경 반영)

```
helm upgrade my-app ./mychart --set replicaCount=5
```

■ 릴리즈 변경 이력 보기

```
helm history my-app
```

REVISION	STATUS	CHART	DESCRIPTION
1	superseded	mychart-0.1.0	Install complete
2	deployed	mychart-0.1.0	Upgrade complete

■ 이전 버전으로 롤백

```
# 직전 버전으로 롤백
```

```
helm rollback my-app
```

```
# 특정 리비전으로 롤백(helm history release)
```

```
helm rollback my-app 1
```

□ Helm은 모든 변경 이력을 K8s Secret에 저장 → 언제든지 되돌릴 수 있다

9.6 실습

■ 실습 시나리오 : nginx Chart를 만들어 설치 → 수정 → 롤백 → 삭제

1. Chart 생성

```
helm create mynginx
```

2. values.yaml 수정 (replicaCount: 1 → 2)

3. 미리보기 & 검증

```
helm lint mynginx
```

```
helm template mynginx | head -50
```

4. 설치

```
helm install web ./mynginx
```

5. 업그레이드 → 6. 롤백 → 7. 삭제

```
helm upgrade web ./mynginx --set replicaCount=4
```

```
helm rollback web 1
```

```
helm uninstall web
```